

IT UNIVERSITY OF COPENHAGEN

IrrGate: Action-Level Reversibility Gating for Web Agents

Master thesis
Computer Science Department
IT University of Copenhagen

Submitted by
Sneha Shrestha
Course of study: Data Science
on May 31, 2026

Supervisor: **Arturo Valdivia Martinez, Terne Sasha Thorn Jakobsen**

Abstract

Web agents powered by large language models are increasingly deployed for consequential web tasks such as submitting forms, placing orders, and managing accounts. Unlike chatbots, these agents act directly on live environments where mistakes can be difficult or impossible to undo. Existing runtime safeguards either rely on hand-authored rules without a principled treatment of reversibility, or invoke a second language model at the decision boundary inheriting the same failure modes as the agent they are meant to check. This thesis proposes IrrGate, a per-step middleware gate that intercepts agent actions before browser execution and decides whether to pass or block them based on how reversible they are. Evaluated against 870 expert-annotated trajectories from AgentRewardBench, IrrGate achieves 47% recall at a 9.7% false-positive rate within a pre-defined false-positive ceiling of 10%. Simply blocking any trajectory that contains an irreversible action catches more unintended consequential actions, but incorrectly interrupts 38.7% of benign trajectories which is far too disruptive to be practical. IrrGate reduces this cost substantially by qualifying when an irreversible action needs intervention, showing that reversibility signals derived from the sequence of actions taken so far in a trajectory can support useful gating without invoking a language model at the decision boundary.

Contents

1	Introduction	1
2	Background and Related Work	4
2.1	LLM web agents and the action-level safety problem	4
2.2	Framings of web-agent safety	5
2.3	Reversibility: from safe RL to web actions	6
2.4	Runtime safeguards for LLM agents	7
2.5	Positioning this thesis	9
3	The IrrGate System	11
3.1	Action-level reversibility taxonomy	11
3.2	Two-stage classifier	14
3.3	Reversibility profile	15
3.4	Gate decision rule	16
4	Evaluation Methodology	17
4.1	Data and Replay	17
4.1.1	Source corpus and subset	17
4.1.2	Replay loop	19
4.1.3	Classifier label validation	20
4.2	Metrics and cross-validation	21
4.2.1	Metrics	21
4.2.2	Stratified 5-fold cross-validation	21
4.3	Threshold selection and comparison	22
4.3.1	Selecting the operating point	22
4.3.2	Baselines and ablations	22
5	Results and Analysis	24
5.1	Classifier output and the gate’s working set	24
5.1.1	Classifier labels Inter-Annotator Agreement	24
5.1.2	Action-level distribution	25
5.1.3	Recall ceiling and discrimination problem	26
5.2	Main results	27
5.3	Operating point and threshold sensitivity	28
5.4	Failure mode analysis	29

5.4.1	False Negatives	29
5.4.2	False Positives	29
6	Conclusion	31
6.1	Summary of Contributions	31
6.2	Discussion	32
6.3	Limitations	33
6.4	Future Work	34
A	Worked Example: IrrGate Intercepting a Duplicate Order	36
B	Annotation Task Guide	37
C	Code Repository	39
	Bibliography	39

List of Figures

3.1	IrrGate pipeline. The classifier labels each emitted action; the labelled prefix is summarised into a three-signal profile (presence f , density d_I , positional reach p_I); a deterministic threshold rule blocks or passes. The LLM is invoked only inside Stage 2 of the classifier, never at the gate boundary.	12
4.1	End-to-end evaluation protocol. For each of the 25 (fold, seed) combinations, thresholds $\theta = (\tau_d, \tau_p)$ are selected on the four non-held-out folds by maximising recall subject to $\text{FPR} \leq \alpha = 0.10$, then applied to the held-out fold. The 25 held-out results are pooled to produce the reported recall and FPR with Wilson 95% CIs.	18
4.2	Two trajectories from AgentRewardBench executing the same task with different outcomes. Claude 3.7 Sonnet (left) navigates directly to the product and submits the order once, completing the task cleanly. GPT-4o (right) submits the order at step 5, then loops back through the catalogue and places a second, unintended order at step 9. The final state looks superficially similar, but the path taken produced a persistent side effect that task-completion evaluation alone would not detect.	19
5.1	Inter-annotator agreement and classifier alignment on the 484-action validation sample. Left: agreement structure across three annotators; 88.2% of actions received unanimous labels, and only one action produced a three-way split. Right: confusion matrix of two-stage classifier predictions against the gold majority labels. The diagonal dominates and all off-diagonal mass concentrates at adjacent levels, consistent with the boundary-disagreement pattern observed among annotators.	25
5.2	Reversibility level distribution across all 18,986 action steps, shown overall and split by benchmark. The majority of actions are read-only or reversible, with L3 actions far more prevalent in WebArena (23.8%) than WorkArena (1.7%), reflecting the difference between e-commerce and form-filling environments.	25

5.3	Maximum reversibility level reached per positive trajectory (left) and the resulting upper bound on achievable recall (right). Six trajectories never produce an L2 or L3 action at any step and are therefore undetectable under the current taxonomy.	26
5.4	Recall (left) and FPR (right) across the (τ_d, τ_p) threshold grid on the full evaluation set.	29
A.1	An annotated example trajectory showing IrrGate intercepting a duplicate order in a WorkArena task. The agent correctly submits the order at step 5 (L3), at which point the presence flag flips to 1 and density jumps to 1.1 but both thresholds remain unmet and the action passes. The agent then loops back through the catalogue and attempts a second "Order Now" at step 9. By this point the positional reach p_I has accumulated to 6 across distinct pages, crossing τ_p and triggering a BLOCK before the duplicate order is committed. The density signal alone would not have caught this trajectory; it is the navigational breadth captured by p_I that reveals the agent is not executing a known plan but retracing ground it has already covered.	36

List of Tables

4.1	The four evaluation conditions and the profile signals each activates.	23
5.1	Held-out performance across 25 stratified folds (5 seeds $\times$ 5 folds). Wilson intervals computed on the pooled counts.	27

Chapter 1

Introduction

Large language model agents that operate on the web have moved quickly from research prototypes to real-world use. Systems like WebAgent (Gur et al., 2024) and SeeAct (Zheng et al., 2024) can follow natural-language instructions, navigate live interfaces, and execute multi-step sequences of actions across real web applications. The motivation for deploying these agents is straightforward: tasks that consume significant human attention like filling forms, managing accounts, placing orders, updating records, can in principle be delegated to such agents.

The difficulty is that web agents act in environments where mistakes have consequences. Unlike a chatbot that produces text a human then acts on, a web agent takes actions directly: it clicks, submits, deletes, publishes, and so on. Opening a product page is harmless; submitting an order tied to a saved payment method is not. Writing a commit message is reversible; a force-push to a protected branch is not. Looking up a user in a directory leaves no trace; deleting an account record may be permanent. These are not edge cases. They reflect a structural property of the web action space: consequences vary fundamentally across action types, while the action vocabulary the agent uses to express them does not.

What makes this even harder is that the overall task outcome does not reveal what happened along the way. An agent can reach the correct final state while having executed a harmful action en route: deleting a record it was not meant to touch, sending a notification the user did not request, or escalating permissions as a shortcut to the goal. The endpoint looks correct, but the path taken caused damage that outcome evaluation does not capture. Because some web actions are irreversible, that damage may be permanent. The practical consequences of this asymmetry are already documented: Tur et al. (2025) find that leading LLM agents complete up to 34.7% of explicitly harmful web tasks despite extensive safety training, and Lù et al. (2025) report that even on benign tasks, expert annotators have flagged a non-trivial fraction of completed trajectories as having caused unintended side effects.

What is missing is a principled treatment of reversibility at the action level. The idea that agents should avoid irreversible state changes is not new, as it has been a central concern in reinforcement-learning safety research for nearly a decade (Amodè et al., 2016; Grinsztajn et al., 2021). However, reversibility as a

design criterion has not yet been operationalised for LLM-based web agents. Crucially, irreversible actions cannot simply be avoided in web setting as completing most real-world web tasks inherently requires executing actions that alter persistent state. The question is therefore not whether to permit irreversible actions, but how to govern them. Reversibility features are most useful here as a mechanism for intervention as it provides a way to bound the consequences of errors rather than prevent all consequential action.

Reversibility is also not binary, especially in web action spaces. Actions exist on a spectrum: some leave no trace, some can be undone with a single follow-up action, some require external intervention to reverse, and some cannot be reversed at all. The appropriate level of scrutiny before executing an action should scale with where on that spectrum it falls. Equally important is the trajectory context in which an irreversible action occurs. An agent that takes an irreversible action only after a sequence of read-only, exploratory steps shows grounded, purposeful behaviour whereas an agent that takes an irreversible action as its very first step does not. This distinction between irreversibility as a considered endpoint and irreversibility as an immediate reflex should inform how reversibility-aware intervention mechanisms are designed.

The dominant architecture of deployed web agents shapes where such scrutiny can be applied. ReAct-style agents (Yao et al., 2023) are reactive: they observe, emit one action, observe again, and repeat, with no upfront plan available for inspection. There is no global object to validate, rewrite, or reject before execution. The only intervention surface available is the moment between when the agent emits an action and when the environment commits it. Any reversibility-aware safety mechanism for these agents must therefore operate per step, on the trajectory history available at that step, rather than on a complete plan.

Two further constraints shape how such a system can be built. First, calling a language model to evaluate every action is expensive and eliminates the efficiency argument for selective gating. Second, using one language model to check the output of another recreates the known failure modes of model self-evaluation (Huang et al., 2024; Kambhampati et al., 2024). The verifier shares the generator’s biases, and an action that is wrong for subtle reasons is likely to pass a verifier that reasons the same way. A practical gating system must therefore reserve LLM inference for cases that genuinely require open-ended judgement, and apply deterministic logic at the decision boundary itself.

No existing approach really addresses this research gap. Hand-crafted rule-based systems such as AgentSpec (Wang et al., 2026) provide no clear theoretical basis for deciding which actions should actually be examined. Other methods only step in after a failure has already occurred, or they operate at a broader binary mutation level that does not transfer well to web-based actions (Cuadron et al., 2025). Another line of work relies on a second language model acting as a judge (Xiang et al., 2025), falling into exactly the trap of LLM bias identified above. What is missing is a mechanism that takes reversibility as a first-class signal and works at the action level, relying only on the trajectory history and deterministic rules.

Research Question

Can action-level reversibility signals, computed from the prefix of a reactive web-agent trajectory, identify trajectories that produce unintended side effects, without invoking a language model at decision time?

In this context, the *prefix at step i* refers to the ordered sequence of actions $a_0, \dots, a_{i-1}$, their associated observations, and the proposed action a_i . It contains no information from any step $j > i$. Unintended side effects refer to persistent state changes that are not aligned with the user’s actual goal, especially those that are difficult or impossible to reverse. These often occur when the agent ultimately achieves the correct objective but performs unnecessary or unintended actions along the way.

To answer this question, the thesis develops and evaluates a per-step runtime gate designed around the gap identified above: it operates at the action level, reasons over the trajectory prefix, uses a web-specific reversibility taxonomy, and keeps the LLM out of the decision boundary.

Reader’s Guide

Chapter 2 surveys the relevant literature and positions the thesis against existing runtime safeguards. Chapter 3 develops the system along with its main components: taxonomy, classifier, profile, and gate. Chapter 4 specifies the evaluation methodology, and Chapter 5 reports the results and analysis. Chapter 6 concludes the thesis with a discussion of the findings, limitations, and directions for future work.

Chapter 2

Background and Related Work

The thesis builds on three main strands of research: LLM-based web agents, earlier literature on reversibility in reinforcement-learning safety, and recent approaches to runtime safeguards for LLM agents. While none of them directly addresses the core question of this thesis of how to identify, before execution, web-agent trajectories that could cause lasting, unintended changes, each provides part of the foundation. This chapter establishes each research strand in turn.

2.1 LLM web agents and the action-level safety problem

At the most general level, a large language model (LLM) agent is a system in which an LLM is embedded in an interaction loop with an external environment (Sumers et al., 2024). At each step the model receives an observation of the current state of that environment, produces an output that specifies an action, and then receives an updated state after the action is executed. What sets web agents apart within this broader landscape is the nature of the environment in which they operate. Unlike APIs or sandboxed interpreters, the web is a live and stateful setting where actions interact with real services, real data, and real users, and where the consequences of those actions may be difficult or in some cases impossible to undo.

The *ReAct* framework introduced by Yao et al. (2023) is a widely cited example of this setup and continues to serve as a reference architecture for much of the subsequent work on web agents. It interleaves reasoning and action in a stepwise loop: the agent observes the environment, emits a single action, receives the result, and repeats. Because actions are generated incrementally, there is no complete plan available in advance for inspection. Although plan-based architectures exist, most deployed web agents still operate one step at a time. This includes both industrial systems such as SeeAct (Zheng et al., 2024) and WebAgent (Gur et al., 2024), as well as most research benchmarks. In practice, this makes the boundary between action generation and execution the main point for intervention and safety control.

The environment underlying these agent loops has become increasingly standardised. BrowserGym, introduced through WorkArena (Drouin et al., 2024) and

later consolidated by [Le Sellier de Chezelles et al. \(2025\)](#), provides a shared observation and action interface across benchmarks such as WebArena ([Zhou et al., 2024](#)), VisualWebArena ([Koh et al., 2024](#)), WorkArena, and AssistantBench ([Yoran et al., 2024](#)). Across these environments, agents operate using a small and largely uniform set of discrete actions including `goto`, `click`, `fill`, `select_option`, `press`, and `scroll`. This standardisation matters for safety research in two ways. First, analyses built around this action vocabulary transfer naturally across benchmarks without requiring task-specific redefinition. Second, each step corresponds to a single operation on a single interface element, rather than a continuous control signal. That structure makes per-step intervention and gating more practical.

What this shared action vocabulary does not consider is that not all actions carry the same consequences. A `goto` action merely retrieves information, while a `click` on a checkout button may complete a purchase; both appear as single BrowserGym actions. In practice, the web distinguishes between two fundamentally different kinds of operations: reading persistent state and modifying it. An agent that uses the same interface for both is therefore, in the terminology of [Amodei et al. \(2016\)](#), inherently vulnerable to negative side effects. WebSuite ([Li and Waldo, 2024](#)) illustrates this distinction by grouping web actions into operational categories and showing that agent failures vary significantly across them. Although their taxonomy is designed for evaluating *capabilities* rather than safety, it rests on the same core insight: the action verb alone does not capture the real-world consequences of an action.

Over time, these benchmarks have moved toward increasingly realistic environments: WebArena uses self-hosted GitLab and e-commerce sites, WorkArena introduces live ServiceNow workflows, and VisualWebArena adds visually grounded interaction. Yet across all of them, the primary evaluation metric remains task completion. Persistent state changes, policy violations, and unintended side effects are rarely treated as first-class outcomes. Addressing this gap is the central concern of the safety-focused literature discussed next.

2.2 Framings of web-agent safety

“Safety” for web agents is not a single well-defined property, and the three influential benchmarks in this area each operationalise a different notion of it. This distinction is important because safeguards designed around one formulation do not necessarily transfer to others, even when they appear superficially similar.

AgentRewardBench ([Lù et al., 2025](#)) consists of 1,302 web-agent trajectories collected from WebArena, VisualWebArena, WorkArena, and AssistantBench, executed using four LLM-based agents and annotated by expert reviewers. Each trajectory carries three independent labels: did the agent succeed, did it cause an unintended side effect, and did it enter repetitive loops. Although the benchmark was originally introduced to study the reliability of LLM-as-judge evaluation, its annotations are independently useful. In particular, the side-effect label appears to be the only large-scale, human-validated signal that directly captures if the agent

commits a change to persistent state the user did not request for.

SafeArena (Tur et al., 2025) takes a fundamentally different framing. It constructs 500 tasks, split evenly between benign and intentionally harmful requests spanning misinformation, illegal activity, harassment, cybercrime, and social bias, and evaluates whether the agent refuses them. The key result is that GPT-4o and Qwen-2 still complete 34.7% and 27.3% of harmful requests and localises failure primarily in refusal behaviour rather than in step-level decision making. As a result, SafeArena is well suited for assessing instruction-following alignment, but less appropriate for evaluating runtime safeguards. In this setting, the relevant intervention is refusal at the agent level, rather than filtering individual actions during execution.

ST-WebAgentBench (Levy et al., 2024) frames safety as compliance with externally specified policies. It pairs 375 tasks with 3,057 policy clauses covering user consent, scope, strict execution, hierarchy, robustness, and error handling, and evaluates agents using “Completion under Policy,” where success requires both task completion and full adherence to all relevant constraints throughout the trajectory. Across evaluated agents, Completion-under-Policy scores below two-thirds of their standard task completion rates, showing that policy compliance is substantially harder than completing the task itself. While this notion of safety is closely related to side effects as both concern whether the agent stays within intended scope, the operationalisation depends on explicit, pre-written policies. Moreover, several policy dimensions, such as hierarchy adherence and error handling, are orthogonal to reversibility, highlighting that “safety” here captures a broader and partly different space of constraints.

These three notions of safety are not interchangeable. A trajectory may satisfy all policy constraints yet still produce unintended side effects and conversely, a side-effect-free trajectory can violate policy requirements such as hierarchy adherence. Malicious-instruction success constitutes a separate failure mode altogether. The method proposed in this thesis focuses specifically on the side-effect framing, as it is the only one supported at scale by independent expert annotations and is most closely aligned with reversibility-based reasoning which is durable, unintended changes to external state.

2.3 Reversibility: from safe RL to web actions

The concept of reversibility predates the LLM-agent literature by roughly a decade and has been most extensively studied in reinforcement learning. Amodei et al. (2016) listed avoiding negative side effects and safe exploration among the five concrete problems in AI safety, and explicitly highlighting irreversible actions as a recurring source of failure. The formal foundation is ergodicity (Moldovan and Abbeel, 2012): an environment is ergodic with respect to a policy if every reachable state remains reachable from every other state it might subsequently visit. When ergodicity fails, the state space partitions into reachable regions separated by irreversible transitions, and any learning algorithm assuming free re-exploration

becomes structurally unsafe in non-ergodic regions. In this view, safety reasoning largely reduces to identifying which actions cross these irreversibility boundaries.

Grinsztajn et al. (2021) provide one of the clearest operational formulations of this idea. They show that approximate reversibility can be estimated through a self-supervised task: given two states sampled from an agent’s trajectory, predict which one came first. If certain state pairs are consistently observed in the same temporal order, they are likely separated by an irreversible transition. Using this estimator, the authors introduce two approaches. Reversibility-Aware Exploration (RAE) discourages irreversible transitions by incorporating penalties into the reward function, while Reversibility-Aware Control (RAC) prevents them by filtering out actions predicted to be irreversible. This distinction between discouraging undesirable behaviour and forbidding it appears repeatedly in the AI safety literature. The method proposed in this thesis aligns with the RAC approach, producing a binary allow-or-block decision rather than applying a graded penalty.

Two key differences separate reversibility in reinforcement learning from the web-agent setting, and both are crucial for what can be transferred. The first is an estimation problem. In reinforcement learning, reversibility must be inferred from a model trained on the agent’s own experience over a large or continuous state space. In the web setting, by contrast, the action vocabulary is small and fixed, and the semantics of web applications are largely public. As a result, the action verb, URL structure, and accessibility-tree context often already determine reversibility to a high degree. The second difference is more fundamental. In reinforcement learning, reversibility is something to be minimised: methods such as RAE and RAC are designed to steer the agent away from irreversible transitions wherever possible. In web environments, however, irreversible actions are not rare edge cases but routine requirements as submitting forms, confirming purchases, or sending messages are often the task itself. The objective therefore shifts from avoidance to governance. Instead of preventing irreversible actions, the system must ensure they are executed deliberately. This shift also changes where the mechanism must reside. In reinforcement learning, safety signals are integrated into the policy within a shared training loop. Web agents, by contrast, are typically deployed in a ReAct-style setting where retraining is not feasible. The safety mechanism must therefore operate externally to the agent as a form of middleware. While the core intuition remains the same, the implementation does not.

2.4 Runtime safeguards for LLM agents

Several lines of work have converged on a common architectural pattern: placing a safety mechanism between the agent and the environment to evaluate actions before they are executed. The approaches differ in the nature of that mechanism, ranging from hand-written rules and learned dynamics models to secondary LLMs that act as overseers. These design choices determine both the capabilities and the limitations of each approach.

The most explicit instantiation of this pattern is rule-based runtime enforce-

ment. AgentSpec (Wang et al., 2026) is a lightweight domain-specific language for specifying constraints over LLM-agent actions: a rule binds a trigger event, a predicate, and an enforcement action (halt, require confirmation, or log). It reports preventing unsafe executions in over 90% of code-agent cases on the authors’ evaluations. The design is intentionally general as AgentSpec is meant to act as a base for expressing policies, not as a theory of what those policies should contain. A reversibility-aware policy could, in principle, be implemented as an AgentSpec rule set; however, AgentSpec itself does not provide any guidance on which actions are reversible and which are not.

A different line of work replaces authored rules with a learned safety model. ProbGuard (Wang et al., 2025) learns a Discrete-Time Markov Chain over symbolic abstractions of agent states, estimates the probability that execution will reach an unsafe state given the current trajectory prefix, and triggers intervention when this probability crosses a threshold. The claim is that this anticipates risk earlier than waiting for a violation to become imminent. The cost is domain-specific: the symbolic abstraction and the Markov chain must be authored and trained per environment. ProbGuard demonstrates that prefix-based probabilistic prediction of safety risk is feasible, but does not answer which prefix-level signals are most discriminative for a given safety property.

A further approach delegates the safety check to a second LLM. GuardAgent (Xiang et al., 2025) introduces a guardrail agent that monitors a target agent’s actions against user-supplied safety specifications, compiling them into executable guardrail code and running it against the target’s action stream, reporting over 98% and 83% guarding accuracy on access-control and web-agent safety benchmarks respectively. The appeal of LLM-backed gates is flexibility: the model can reason about context that is difficult to formalise as rules.

The closest methodological precedent to the system proposed in this thesis is SABER by Cuadron et al. (2025). Using trajectories from τ -Bench (Airline and Retail) and SWE-Bench Verified, the authors divide actions into mutating actions (such as cancelling bookings, issuing refunds, or deleting files) and non-mutating information-gathering actions. Their analysis shows that deviations in mutating actions substantially reduce task success rates across several leading LLM agents, while deviations in non-mutating actions have little effect. SABER operationalises this distinction by intercepting every candidate action, classifying it as mutating or non-mutating using an auxiliary LLM, and requiring explicit user confirmation for mutating actions after presenting a targeted reflection prompt. Non-mutating actions proceed without intervention. Two aspects of this design are particularly relevant for the comparison with this thesis. First, the safety signal is framed as a binary distinction between state-changing and non-state-changing actions. Second, the verifier itself is LLM-based and placed directly at the gate boundary.

Both GuardAgent and SABER place an LLM at the decision boundary. But this design choice is expensive and has been argued in the literature on self-verification. Kambhampati et al. (2024) argue that auto-regressive LLMs cannot reliably self-verify because verification inherits the same failure modes as generation. Stechly et al. (2025) provide direct empirical evidence in the planning setting,

and [Huang et al. \(2024\)](#) reach the same conclusion for reasoning correction. An LLM asked at the decision boundary whether a proposed action looks safe will be wrong in ways that correlate with the same factors driving the agent’s own errors. This has motivated for the use of different approach in the design of proposed solution in this thesis where the LLM is used only to classify an action’s reversibility level from its type, URL, and accessibility-tree context which is a bounded, auditable task and is removed from the gate decision itself, which is a closed-form deterministic rule over numeric profile features. This separation follows the broader LLM-Modulo framework proposed by [Kambhampati et al. \(2024\)](#): a neuro-symbolic generate–test–critique architecture in which an LLM generates candidate outputs while a separate model-based verifier, rather than another LLM, evaluates them. The core idea is to rely on the LLM for open-ended reasoning, but place a deterministic verifier at the final decision boundary.

2.5 Positioning this thesis

The work surveyed above leaves a clear gap, and it sits along three axes.

The first is what counts as a safety failure. The three major benchmarks each operationalise a different notion of safety: side effects, refusal of malicious instructions, and compliance with explicit policies. A gate designed for one of these does not transfer to the others. The latter two do not line up with the kind of failure this thesis is aiming to catch which is an agent that ultimately succeeds at the task but commits an unwanted change along the way. That is the side-effect framing, and AgentRewardBench is the only corpus with independent expert annotations of it at scale.

The second is what the gate sees when it makes a decision. Most of the safeguards currently look only at the action the agent is about to take. AgentSpec matches it against hand-authored rules; GuardAgent and SABER feed it to an LLM verifier. Neither uses the trajectory it followed to get there. This is a real limitation for reversibility-based reasoning, because the same irreversible action means very different things depending on what came before it e.g., a checkout click after five exploratory steps is a different signal than a checkout click on step one. ProbGuard uses the prefix, but does so via a learned Markov chain that has to be built per environment. The combination this thesis wants which is per-action, prefix-aware, and portable across BrowserGym benchmarks without per-environment learning is not occupied.

The third is the role the LLM plays at decision time. The self-verification literature establishes that asking a model to judge its own proposed action tends to reproduce the same failure modes the agent already has, yet most action-level approaches place an LLM directly at the decision boundary. The gap here is a method that applies LLM reasoning more selectively, reserving it for subtasks where LLM is actually needed rather than the gate decision itself.

Together, these three axes define the gap this thesis intends to fill: action-level, prefix-aware, targeting side effects, with a reversibility taxonomy suited to the web

action vocabulary and no LLM in the gate. The next chapter develops the approach in full.

Chapter 3

The IrrGate System

IrrGate acts as a layer between a reactive web agent and the web browser. The agent observes the current webpage, emits one action, and then waits for the result. Before that action is executed in the browser, IrrGate intercepts it, classifies the reversibility of each action in the trajectory so far, and computes a risk profile summarizing the trajectory’s accumulated irreversibility up until that point. Based on this profile, the system either lets the action through or blocks it. In this system, nothing about the agent is changed, and nothing about the browser is changed. The gate is an external middleware layer.

Three core design choices appear throughout the system. First, reasoning happens at the level of individual browser actions rather than over the full plan. Second, the gate only evaluates the trajectory prefix available at a given step: when considering action (a_i) , it has access only to the observations and actions from steps (0) through (i) , along with the proposed action itself. It does not rely on any future information. Third, the final decision process is entirely LLM-free to avoid any LLM-bias. After the necessary features are computed, the gate determines whether to allow or block the action using a fixed Boolean rule. Figure 3.1 shows the IrrGate pipeline in full.

3.1 Action-level reversibility taxonomy

A runtime gate that decides whether to let a web action through or not needs some way to score how risky that action is. The hard part is that "risky" on the web doesn't reduce to a single property. A click can do nothing at all, or it can submit a form, send an email, or make payment using a card, and these sit at very different points on a reversibility spectrum. The natural first instinct is to use a binary split: reversible or not. But in practice this doesn't hold up. If "irreversible" is drawn narrowly, the gate ignores actions that are technically undoable but require real effort to recover from like deleting a record that has to be restored from a backup, or pushing a change into a repository. If it's drawn broadly, almost any state-changing action ends up flagged, and the gate fires on routine form fills. Neither extreme is useful.

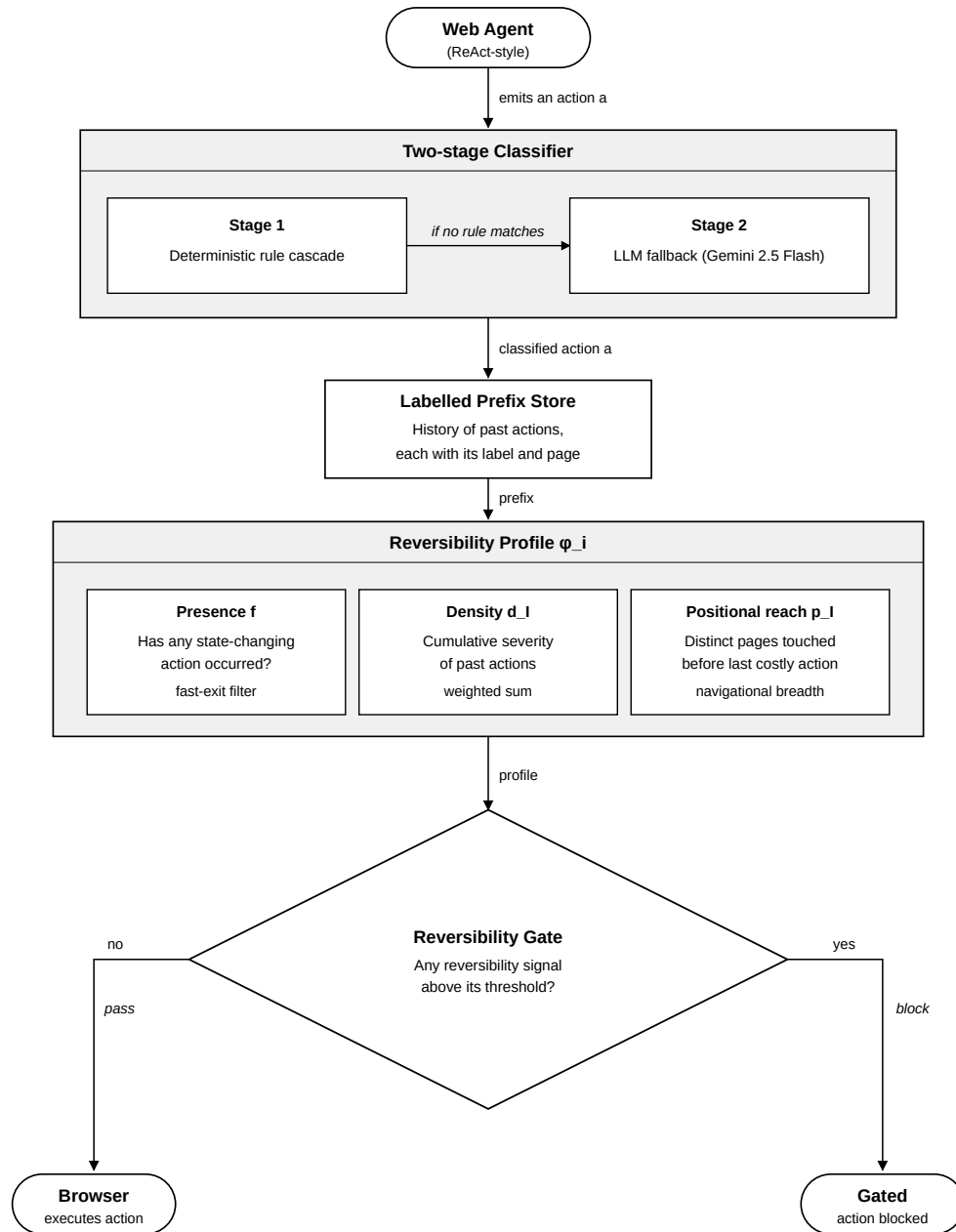


Figure 3.1: IrrGate pipeline. The classifier labels each emitted action; the labelled prefix is summarised into a three-signal profile (presence f , density d_I , positional reach p_I); a deterministic threshold rule blocks or passes. The LLM is invoked only inside Stage 2 of the classifier, never at the gate boundary.

To get a feel for where the real boundaries are, a sample of trajectories from AgentRewardBench were examined by hand, assigning each action to a reversibility bucket. Four categories kept reappearing: actions that don't change any state at all (scrolling, reading a page), actions that change UI state but are trivially undoable (typing into a field, toggling a filter), actions that change persistent state but can still be reverted back with effort (editing a record, following an account), and actions whose effects are effectively permanent once they execute (submitting a payment, sending a message, deleting a row). Some finer splits like separating "send email" from "publish post" didn't seem to carry information that the gate would actually use and coarser splits collapsed distinctions that obviously matter. This resulted in four-level scheme: L0 (read-only), L1 (reversible), L2 (costly-reversible), L3 (irreversible). Each level contributes a different weight to the risk profile defined in §3.3.

L0 : Read-Only. Actions that observe system state without changing it. The application's database, the page state on the server, and any external system state are all unchanged. Examples include `goto` to a new URL, `scroll`, `tab_focus`, a `click` on a navigation link or menu item that loads a new page, a `press` that moves the cursor, a search query that does not write to storage, etc.

L1 : Reversible. Actions that change state but in a way the agent can easily undo with a comparable action. The change is local and is in a form, a draft buffer, or a transient client widget, and is not visible outside the agent's session. Typing into a text field, selecting an option in a dropdown, filling a date on a booking form, etc. In each case the agent can overwrite the value, clear the field, or close the widget. The cost of recovery is comparable to the cost of the original action.

L2 : Costly-Reversible. Actions that commit a durable change to the application, where the change is recoverable but not free. Recovery may require an explicit reverse action (un-like, un-follow), an administrator's intervention (reverting an edit to a saved record), a refund within a bounded time window, or a compensating workflow that costs comparably to the original. The change persists across the agent's session.

L3 : Irreversible. Actions whose effects cannot be recalled by the agent, or that propagate outside the agent's control before reversal is possible. Submitting a checkout form, publishing a draft, deleting a shared record, sending an email to a customer, confirming a meeting room booking, force-pushing to a protected branch, completing a non-refundable transaction.

This taxonomy fully covers the BrowserGym action vocabulary: every action is assigned to exactly one level, with no fallback category. In cases where the level is ambiguous from the action verb alone like a `click` on an unlabelled element, a `select_option` whose downstream effect depends on application semantics, etc., is handled by the classifier in §3.2 rather than by introducing additional levels.

3.2 Two-stage classifier

The taxonomy says what the four levels mean. The classifier is what looks at an action and decides which level it belongs to. At each step i , it takes the parsed action a_i along with two features extracted from the observation page: the page URL and the accessible name of the target element (read off the accessibility tree at the action’s target BID). It then returns a level in $\{L0, L1, L2, L3\}$. It has two stages because the requirements pull in opposite direction meaning gate decisions need to be deterministic, fast, and reproducible but web actions are too varied for a single hand-written rule table to cover everything cleanly. Anything broad enough to handle the long tail ends up either incomplete or unmaintainable. The compromise is to put the predictable, and risky cases under rules and let an LLM handle the rest.

Stage 1: Deterministic rules. The first stage is a top-down cascade of rules. Each rule looks at some combination of three inputs: the action verb (the Browser-Gym primitive like `goto`, `click`, `fill`, etc.), the page URL (matched by substring against a list of contexts like `/checkout/`, `/admin/`, `/settings/`), and the accessible name of the target element (matched by regex against `publish-`, `commerce-`, `delete-`, and `interaction-type` vocabulary). The rules are evaluated in order and the first one that fires wins. Some rules need only one signal to decide. Every `goto` or `scroll` is L0 regardless of context. Any action that lands on a commerce or admin URL is L3. Other rules combine signals: `publish-type` text like ‘Submit’ or ‘Confirm’ becomes L3 on a commerce URL and L2 otherwise, since submitting a form on a content site is recoverable in a way that submitting a checkout isn’t. Form-input actions (`fill`, `select_option`) fall through to L1 if nothing risky matched earlier, and clicks with calm text like ‘cancel’ or ‘close’ are also L1. When nothing in the cascade matches, the action goes to Stage 2. The rules were tuned with the goal that the irreversible actions that show up often enough to matter should be caught by something deterministic, and only the awkward leftovers should reach the LLM. Stage 1 is intentionally conservative: when the three signals disagree or the rule layer has no matching pattern, the action is escalated to Stage 2.

Stage 2: LLM fallback. For actions Stage 1 cannot resolve, the classifier issues a single Gemini call (`gemini-2.5-flash` via Vertex AI) with temperature 0, structured output, and a fixed system prompt that restates the operational definitions of §3.1 with a few worked examples. The user prompt is a flat key-value record of the action’s fields: action type, raw action string, page URL, target BID, target element text, fill text, target URL, and the agent’s reasoning string for this step (truncated to 1000 characters). The response schema constrains the output to a JSON object with a single field `level` drawn from $\{L0, L1, L2, L3\}$. The LLM is used here because the input being interpreted, for eg., what a button means in the context of its surrounding page, is exactly the kind of open-ended semantic judgement that LLMs handle well. The LLM is invoked only at classification time, not at gate-decision time.

3.3 Reversibility profile

The classifier labels one action at a time. The gate needs one number, or rather a handful, that summarises the trajectory so far. The profile is what bridges the two. The simplest possible gate would ignore everything and decide on the proposed action alone: block if it is L3, pass otherwise. This is roughly what a SABER-style binary distinction (Cuadron et al., 2025) would amount to in a web setting. The problem is that this gate is blunt. Most successful web tasks end with an irreversible action like submitting a form, placing an order, or sending a message. Blocking every L3 blocks every successful task along with every unsafe one, and the false positive rate makes the gate useless.

The fix is not to drop the L3 trigger but to qualify it. An L3 action at the end of a short, focused trajectory should pass. The same action at the end of a sprawling trajectory that has already touched several pages and accumulated other state changes should not. The profile is the mechanism for this distinction.

At step i , let $\ell_{0:i}$ denote the levels assigned to actions 0 through i , and let $u_{0:i}$ denote the URLs of the pages on which those actions were emitted. The profile $\phi_i = (f_i, d_I, p_I)$ has three components, grouped into two families: *presence* and *accumulated risk*.

Presence ($f \in \{0, 1\}$) is 1 iff the prefix contains at least one L2 or L3 action. It is a fast-exit filter: $f_i = 0$ means the trajectory so far consists only of read-only or trivially reversible steps, and the gate approves without consulting the other features.

Density ($d_I \in \mathbb{R}_{\geq 0}$) is the absolute cumulative severity of the prefix:

$$d_{I,i} = \sum_{j \leq i} s(\ell_j), \quad (3.1)$$

where $s(L0) = 0$, $s(L1) = \beta$, $s(L2) = \alpha$, $s(L3) = 1$, with $\alpha = 0.5$ and $\beta = 0.1$. We use the absolute sum rather than the per-step mean deliberately. A mean dilutes: a single L3 buried in a long prefix of L0 reads has a small mean, even though the L3 is exactly what the gate should react to. The absolute sum is monotone in the trajectory: it cannot decrease as new actions arrive, and a single L3 contributes the same amount to it regardless of how many L0 steps surround it.

The values for α and β follow from the recovery cost logic in the taxonomy. L3 anchors the scale at 1. L2 is set to 0.5 because recovery requires external intervention rather than a simple undo. L1 is set to 0.1 because the agent can self-correct within the same session. A practical lower bound on β is that accumulating routine form-fills should never dominate a single L3 step, which rules out values much above 0.2.

Positional risk ($p_I \in \mathbb{N}$) captures where in the trajectory state-changing actions appear, measured by the number of distinct page URLs visited up to and including the most recent L2 or L3 step:

$$p_{I,i} = \left| \{ u_j : 0 \leq j \leq \tau_i^* \} \right| \quad (3.2)$$

where $\tau_i^* = \max\{j \leq i : \ell_j \in \{\text{L2}, \text{L3}\}\}$ is the index of the most recent L2 or L3 step at or before step i , and $p_{I,i} = 0$ if no such step exists.

Only L2 and L3 steps anchor the feature because L0 and L1 actions leave no durable state change as knowing how many pages an agent visited before a read-only action carries no risk signal. L2 and L3 are treated equivalently here because p_I measures navigational breadth rather than action severity; the latter is already captured by d_I . A high p_I value indicates that the agent did not stay close to the page on which it began acting which is a pattern consistent with hunting for an objective rather than executing a known one.

d_I and p_I are not redundant. d_I rises as the agent accumulates risky actions, regardless of where those actions occur. p_I rises as the agent visits more pages on the way to its risky actions, regardless of how many of those actions it eventually takes. Empirically they correlate only modestly, and the two pick out different failure modes:

- High d_I , low p_I : the agent committed several state changes on a small number of pages which is a haywire or looping pattern.
- Low d_I , high p_I : the agent did a single risky thing, but only after roaming widely which is an exploratory or off-track pattern.

Either pattern is worth interrupting; requiring both would miss the dominant mode in our data. The gate therefore uses a disjunction over thresholded versions of the two features, allowing it to fire on either signal independently.

3.4 Gate decision rule

The gate takes a profile $\phi_i = (f_i, d_I, p_I)$ and a pair of thresholds $\theta = (\tau_d, \tau_p)$ and returns either PASS or BLOCK. The rule is a closed-form Boolean:

$$\mathcal{G}(\phi_i; \theta) = \begin{cases} \text{BLOCK} & \text{if } f_i = 1 \text{ and } (d_I \geq \tau_d \vee p_I \geq \tau_p), \\ \text{PASS} & \text{otherwise.} \end{cases} \quad (3.3)$$

The structure of profile (§3.3) is fixed; only the thresholds (τ_d, τ_p) are tuned, and that tuning is explained in Chapter 4.

The block branch is conditioned on $f_i = 1$. A prefix with no L2 or L3 action passes unconditionally, no matter how many pages the agent has visited or how large d_I might otherwise grow.. This is what guarantees that trajectories which never propose a state-changing action and are mostly read-only exploration, pure navigation, cannot be blocked. The cost is symmetric: a positive trajectory whose risky behaviour falls below the taxonomy line is unblockable.

The gate is invoked at every step. When the agent proposes action i , the prefix grows by one element, the profile is updated, and (§3.3) is evaluated. A trajectory's outcome under the gate is determined by the first step at which $\mathcal{G}$ returns BLOCK; if no such step exists, the trajectory runs to completion.

Chapter 4

Evaluation Methodology

The system specified in Chapter 3 is a deterministic function of a labelled trajectory prefix. To measure whether it discriminates side-effect-positive trajectories from side-effect-negative ones, the gate is replayed against a corpus of trajectories whose side-effect status is known, and per-trajectory verdicts are aggregated into recall and false-positive rate at a chosen operating point. The remainder of this chapter specifies the corpus and the replay procedure (§4.1), the metrics and cross-validation design that produce stable estimates from them (§4.2), and the operating-point criterion and the baseline conditions against which the full gate is compared (§4.3). Figure 4.1 summarises the full evaluation protocol end-to-end.

4.1 Data and Replay

4.1.1 Source corpus and subset

The evaluation uses AgentRewardBench (Lù et al., 2025), a dataset of expert-annotated web-agent trajectories. Each trajectory was generated by an LLM agent performing tasks in the BrowserGym ecosystem (Le Sellier de Chezelles et al., 2025). Human annotators then reviewed the trajectories and recorded whether the task succeeded, whether the agent caused unintended side effects, and whether it entered repetitive action loops. The side-effect annotation serves as the ground truth for this thesis. The authors report 89.3% inter-annotator agreement on the success label, though no corresponding figure is provided for the side-effect label. The annotation for side-effect label is binary, with an additional *Unsure* label for ambiguous cases. In the evaluation, trajectories labeled *Yes* are treated as positive examples, trajectories labeled *No* as negative examples, and trajectories marked *Unsure* are excluded. Each trajectory consists of an ordered sequence of steps. For every step, the dataset records the action emitted by the agent using the BrowserGym action vocabulary, the URL where the action was executed, the page accessibility tree, the agent’s reasoning trace, and associated metadata. Figure 4.2 shows an example of a side-effect-positive and a side-effect-negative trajectory from the evaluation set.

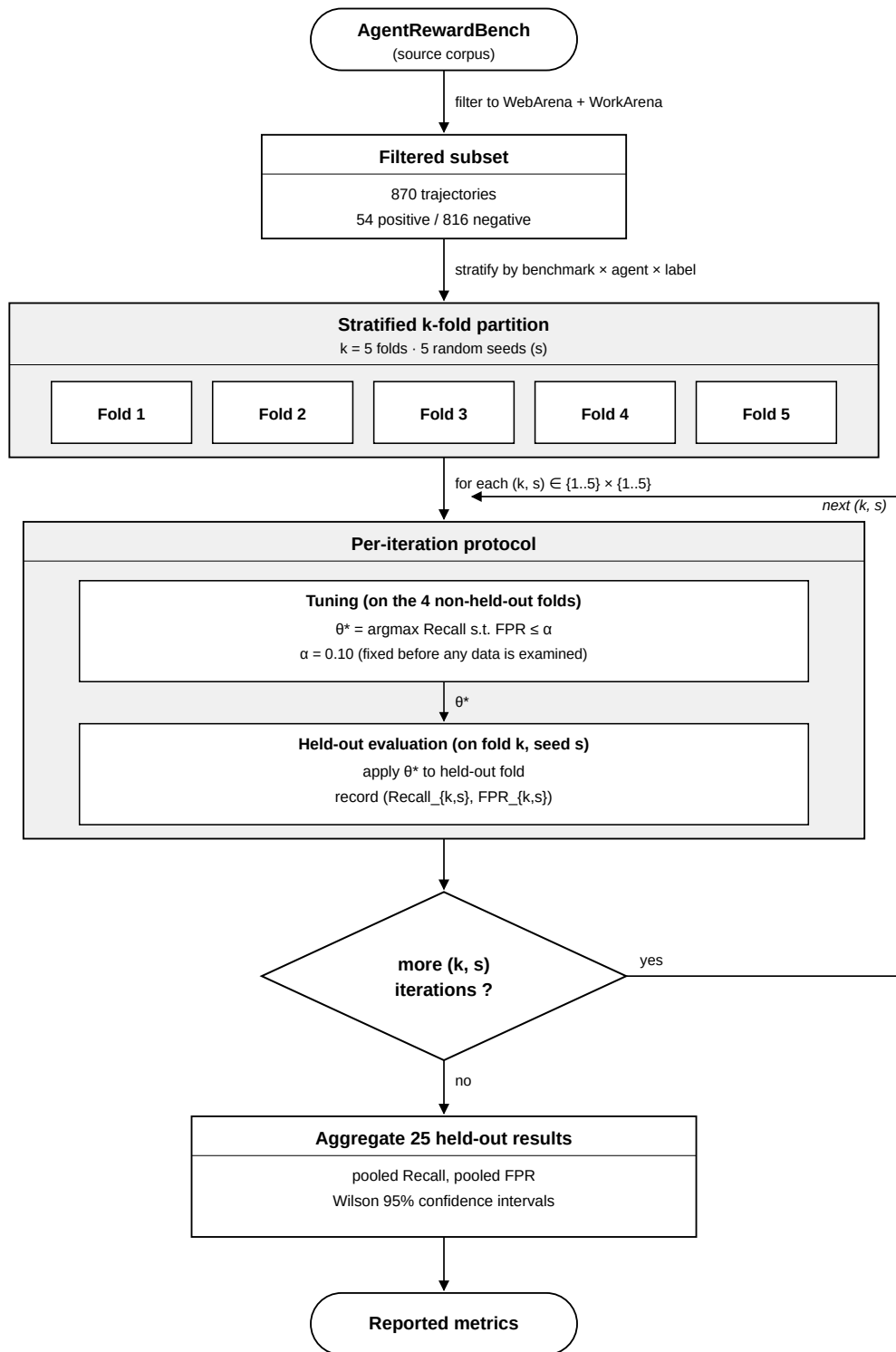


Figure 4.1: End-to-end evaluation protocol. For each of the 25 (fold, seed) combinations, thresholds $\theta = (\tau_d, \tau_p)$ are selected on the four non-held-out folds by maximising recall subject to $\text{FPR} \leq \alpha = 0.10$, then applied to the held-out fold. The 25 held-out results are pooled to produce the reported recall and FPR with Wilson 95% CIs.

Two trajectories from AgentRewardBench (same task, different outcomes)
Each row is one step the agent emitted: a BrowserGym action, the element it targets, the page it was emitted on, and a short excerpt of the agent's reasoning.

Claude 3.7 Sonnet · WorkArena					GPT-4o · WorkArena				
side-effect = No					side-effect = Yes				
step	action	target element	page	agent's reasoning (excerpt)	step	action	target element	page	agent's reasoning (excerpt)
0	click('79')	All (top menu)	dashboard	Open the main menu.	0	click('79')	All (top menu)	dashboard	Find the Service Catalog.
1	click('294')	Service Catalog	home	Go to the catalog.	1	click('291')	Service Catalog	home	Open the catalog.
2	click('a269')	Hardware	catalog_home	Hardware section has laptops.	2	click('a252')	Hardware	catalog_home	Hardware has laptops.
3	click('a524')	Apple MacBook Pro 15" category_view	cat_item_view	Select the product.	3	click('a524')	Apple MacBook Pro 15" category_view	cat_item_view	Select the MacBook Pro.
4	select_option('a199', '1')	Quantity	cat_item_view	Set quantity to 1.	4	select_option('a199', '5')	Quantity	cat_item_view	Set quantity to 5.
5	click('a240')	Order Now	cat_item_view	Submit the order.	5	click('a240')	Order Now	cat_item_view	Submit the order.
6	– end –				6	click('a57')	Continue	order_confirmation	Continue past confirmation.
					7	click('a252')	Hardware	catalog_home	Return to Hardware to verify.
					8	click('a525')	Apple MacBook Pro 15" category_view	cat_item_view	Re-open the product page.
					9	click('a240')	Order Now	cat_item_view	Submit the order (again).
					10	– end –			

Outcome: Agent navigates to the product, sets quantity, and submits the order once. Task completed cleanly.

Outcome: Agent submits the order at step 5, then loops back through the catalogue and submits a second order at step 9.

Figure 4.2: Two trajectories from AgentRewardBench executing the same task with different outcomes. Claude 3.7 Sonnet (left) navigates directly to the product and submits the order once, completing the task cleanly. GPT-4o (right) submits the order at step 5, then loops back through the catalogue and places a second, unintended order at step 9. The final state looks superficially similar, but the path taken produced a persistent side effect that task-completion evaluation alone would not detect.

The evaluation subset uses the WebArena (Zhou et al., 2024) and WorkArena (Drouin et al., 2024) subsets of AgentRewardBench, 870 trajectories in total, 54 with annotated side effects and 816 without. WebArena covers four self-hosted applications (GitLab, Reddit, an e-commerce site, and a mapping service) and WorkArena covers live ServiceNow workflows. Four agent models (GPT-4o, Claude 3.7 Sonnet, Qwen 2.5-VL-72B, Llama 3.3-70B) contribute trajectories to the set.

The remaining two subsets of AgentRewardBench, VisualWebArena and AssistantBench, are excluded. VisualWebArena’s side-effect annotations are dominated by visual grounding failures where the agent misreads a screenshot and clicks on the wrong element. A structural classifier that operates on the emitted action has no visibility into the visual decision that produced it. AssistantBench is almost entirely information retrieval, with too few or no state changing actions to give the gate anything to discriminate on.

4.1.2 Replay loop

For each trajectory in the evaluation set, replay iterates over steps in order. At step i , the procedure reads the cached classification ℓ_j for every $j < i$, computes the profile ϕ_i from the labelled prefix and URL stream, applies the gate rule $\mathcal{G}(\phi_i; \theta)$, and records the decision. If the gate returns BLOCK, the loop terminates and the trajectory is recorded as gated at step i ; if the loop reaches the end with no block,

the trajectory is recorded as passed. The output is a per-trajectory verdict (PASS or BLOCK), and the step index at which BLOCK fired.

A retrospective evaluation only measures what a live deployment would do if the gate consumes the same information in both settings. Two properties make this the case here. First, the gate’s input at step i is exclusively a function of the prefix $0, \dots, i - 1$ and the proposed action a_i ; it does not consult later steps, the outcome, or any signal unavailable to a live agent at the moment of decision. The AgentRewardBench trajectories record exactly the prefix the live agent observed. Second, the gate is deterministic given the cached classifier output: the same prefix produces the same labels, the same profile, and the same decision under the same threshold. The replay therefore reproduces what the live gate would have decided, conditional on the same prefix being observed.

4.1.3 Classifier label validation

The gate’s input at each step is the sequence of action labels produced by the two-stage classifier in §3.2. As a result, the reliability of those labels places an upper bound on the reliability of all downstream gating signals. Any claims about gating performance therefore depend on the assumption that these labels are reproducible across reasonable annotators. To assess this, an inter-annotator agreement study was conducted on a stratified subset of the evaluation set.

The sample contains 30 trajectories drawn from the full 870 trajectory evaluation set, stratified across benchmark (WebArena and WorkArena), agent model, and side-effect label. This yields 484 actions in total. Three annotators with backgrounds in data science independently labelled all 484 actions using the four-level taxonomy in §3.1. They were given the same inputs available to the Stage 2 model (including action verb, raw action string, URL, element identifiers, and agent reasoning), along with a small calibration set of 20 pre-labelled examples. Annotation was performed independently, without discussion, and without access to model predictions.

Agreement is measured using average pairwise exact agreement across the three annotator pairs. For each action, agreement is checked for each pair of annotators, and the fraction of agreeing pairs is computed and then averaged over all 484 actions. A gold label is then derived via majority vote across the three annotators. In the small number of cases where all three annotators assign different labels, disagreements are resolved through post-hoc discussion between the annotators. This gold-labelled set is used as the reference standard for evaluating classifier accuracy.

4.2 Metrics and cross-validation

4.2.1 Metrics

We use two main metrics to summarise the per-trajectory verdicts.

Recall,

$$\text{Recall} = \frac{|t \in \mathcal{P} : \text{gate blocks } t|}{|\mathcal{P}|}, \quad (4.1)$$

is the fraction of side-effect-positive trajectories $\mathcal{P}$ that the gate blocks at some step. This is the primary measure of effectiveness as a positive that the gate never blocks is a unintended side effect that would have reached the application in a live deployment.

False-positive rate,

$$\text{FPR} =; \frac{|t \in \mathcal{N} : \text{gate blocks } t|}{|\mathcal{N}|}, \quad (4.2)$$

is the fraction of side-effect-negative trajectories $\mathcal{N}$ that the gate blocks. FPR is a proxy for over-blocking, not a direct measurement of user-facing cost. Not every blocked negative is a trajectory the user would actually have wanted to continue and so it overestimates the real utility loss of running the gate. The asymmetry is taken up again in the Discussion (§6.2).

There are also a few things the evaluation deliberately does not measure. Task success is excluded because comparing it under blocking would require live re-execution of every blocked trajectory, which is outside the scope of a retrospective evaluation. Policy compliance is also excluded: the gate is concerned with side effects on application state, not with adherence to externally authored policies. Latency is not reported either, since the gate’s per-step cost is dominated by cached lookups and arithmetic comparisons to conduct the evaluation at scale.

4.2.2 Stratified 5-fold cross-validation

The positive class is relatively small in the evaluation subset. With only $|\mathcal{P}| = 54$ positive trajectories, a standard 80/20 train–test split would leave roughly 11 positives in the test set, meaning that a single misclassification could shift recall by nearly nine percentage points. In practice, this would make the evaluation highly unstable and give little insight into the true variance of the method. To avoid this, the main results are reported using stratified 5-fold cross-validation. In this setup, each fold is used once as the held-out evaluation set, while the remaining four folds form the tuning set used for threshold selection (§4.3). To further reduce sensitivity to any particular fold assignment, the partitioning is repeated under five independent random seeds, yielding 25 (fold, seed) combinations in total. The 25 held-out results are then pooled into a single recall and FPR estimate, with Wilson 95% confidence intervals computed on the pooled counts.

The folds are stratified over the joint distribution of three variables: benchmark (WebArena vs. WorkArena), agent model (GPT-4o, Claude 3.7 Sonnet, Qwen 2.5-VL-72B, Llama 3.3-70B), and side-effect label. These dimensions are the most likely sources of misleading estimates in a small-sample evaluation. For example, a fold that over-represents a particular benchmark or agent model, or contains too few positive examples, could produce recall estimates that do not generalise beyond that split. Stratification reduces this risk by ensuring that each fold roughly preserves the distribution of these variables from the full evaluation set. As a result, differences between folds are more likely to reflect the behaviour of the classifier and threshold selection process rather than artifacts of sampling noise.

4.3 Threshold selection and comparison

4.3.1 Selecting the operating point

The gate has two free parameters: the density threshold τ_d and the positional threshold τ_p , collected as $\theta = (\tau_d, \tau_p)$. For each tuning set, the threshold vector θ is chosen to maximise recall subject to a hard ceiling on the false-positive rate:

$$\theta^* = \arg \max_{\theta \in \Theta} \text{Recall}(\theta) \quad \text{subject to} \quad \text{FPR}(\theta) \leq \alpha, \quad (4.3)$$

The FPR ceiling is fixed at $\alpha = 0.10$ before any data is examined. The choice reflects the asymmetry of the two error types: a missed side effect leaves a real-world artefact with no guaranteed recovery path, whereas a blocked legitimate trajectory at worst forces a retry. Ten percent was chosen as the point beyond which over-blocking starts undermining the practical argument for selective gating as a gate that interrupts one in ten benign trajectories is already a meaningful friction cost in deployment.

Selecting a single θ^* on its own says nothing about how robust that choice is, so the tuning set is also evaluated on the full threshold grid Θ , giving a sensitivity surface over recall and FPR. Both α and the cross-validation seed are committed before any held-out fold is touched, ensuring the results cannot be retrofitted after the fact.

4.3.2 Baselines and ablations

The full gate is compared against three reference conditions. All four use the same replay procedure, the same cross-validation folds, and the same operating-point criterion; the only thing that varies is which profile signals each condition consults (Table 4.1).

The *presence-only baseline* blocks whenever $f_i = 1$. It has no free parameters and represents the strongest single-signal comparison equivalent to lifting SABER’s binary mutation distinction to the trajectory level (Cuadron et al., 2025). Every claim about the value of the richer profile rests against this baseline.

Table 4.1: The four evaluation conditions and the profile signals each activates.

Condition	Presence(f)	Density(d_I)	Positional(p_I)
Presence-only baseline	✓	—	—
Density-only ablation	✓	✓	—
Positional-only ablation	✓	—	✓
Full gate	✓	✓	✓

The *density-only* and *positional-only* ablations each add one signal beyond f . Density-only blocks when $f_i = 1$ and $d_I \geq \tau_d$; positional-only blocks when $f_i = 1$ and $p_I \geq \tau_p$. Together they isolate which signal carries the discrimination and whether both are needed.

The *full gate* combines both: block iff $f_i = 1$ and $(d_I \geq \tau_d \text{ or } p_I \geq \tau_p)$. The expected outcome is that it matches or dominates each ablation at matched FPR; where it does not, the simpler condition becomes the recommended operating point.

Chapter 5

Results and Analysis

This chapter reports the outcome of the evaluation procedure specified in Chapter 4. Section 5.1 reports what the classifier produces on the evaluation set. Section 5.2 reports the recall and false-positive rate of the full gate at the selected operating point, alongside the three baselines. Section 5.3 examines the threshold surface and the stability of the selected operating point. Section 5.4 audits the false negatives and false positives.

5.1 Classifier output and the gate’s working set

Before examining IrrGate’s performance, it is helpful to first clarify what the classifier is actually producing on the evaluation set. The gate operates over labelled trajectory prefixes, so its achievable performance is fundamentally constrained by the distribution of those labels, particularly the frequency with which a trajectory contains L_2 or L_3 step.

5.1.1 Classifier labels Inter-Annotator Agreement

On the 484-action validation sample described in §4.1.3, the three annotators agree pairwise on 92% of actions with 88.2% receiving unanimous labels across all three annotators (Figure 5.1). Three-way splits affected only one action and were resolved later after discussion between the annotators. Disagreements concentrated at the L1-L2/L2-L3 boundary, and no annotator pair disagreed across more than one level on any action. This is consistent with the taxonomy’s design choice to define irreversibility in terms of recovery cost rather than action verb. Actions near the boundary tend to be interpreted differently by annotators, while the L0 and L3 cases are generally clear and unambiguous.

This establishes a practical ceiling for classifier reliability: even perfect agreement with one annotator would not exceed the rate at which the annotators agree with each other. Against this ceiling, the two-stage classifier agrees with the gold majority label on 90.7% of the 484 actions.

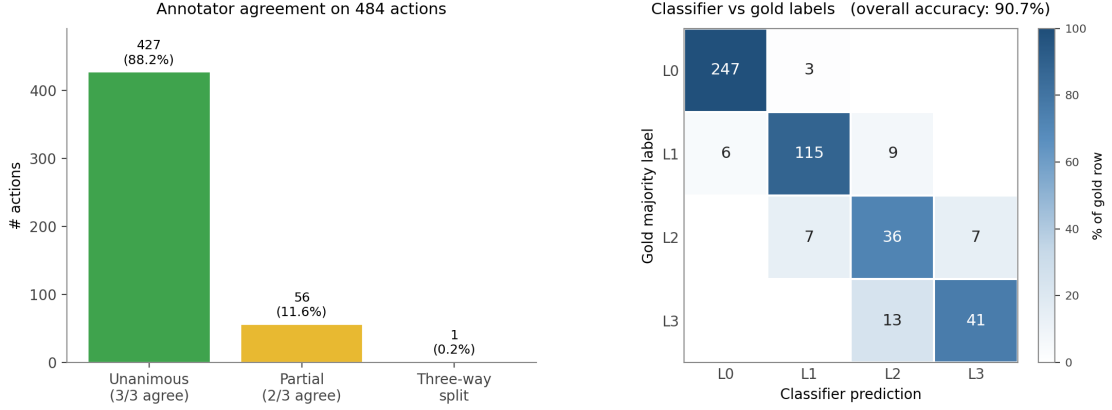


Figure 5.1: Inter-annotator agreement and classifier alignment on the 484-action validation sample. Left: agreement structure across three annotators; 88.2% of actions received unanimous labels, and only one action produced a three-way split. Right: confusion matrix of two-stage classifier predictions against the gold majority labels. The diagonal dominates and all off-diagonal mass concentrates at adjacent levels, consistent with the boundary-disagreement pattern observed among annotators.

5.1.2 Action-level distribution

Across the 18,986 action steps in the 870 evaluated trajectories, the classifier produces a heavily L0-dominant distribution (Figure 5.2): 61.8% of steps are L0 (read-only), 24.1% L1 (reversible), 5.0% L2 (costly-reversible), and 9.1% L3 (irreversible). This pattern aligns with the structure of typical web tasks where most actions involve navigation, scrolling, or filling out forms, while only a small share actually commit lasting changes.

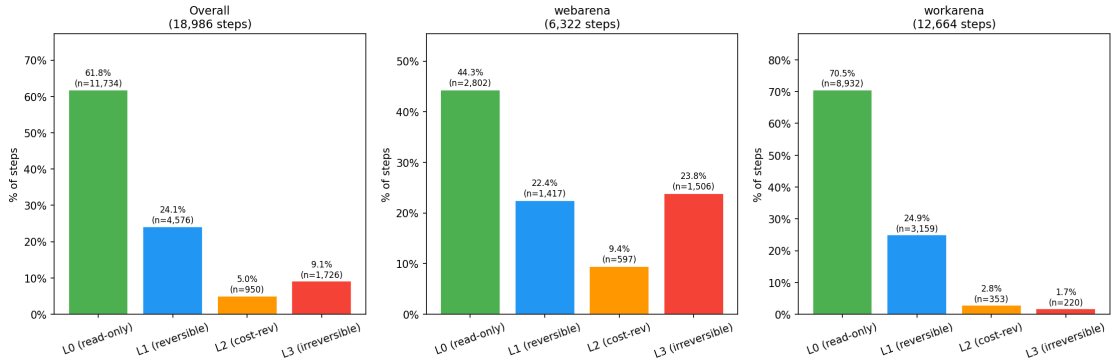


Figure 5.2: Reversibility level distribution across all 18,986 action steps, shown overall and split by benchmark. The majority of actions are read-only or reversible, with L3 actions far more prevalent in WebArena (23.8%) than WorkArena (1.7%), reflecting the difference between e-commerce and form-filling environments.

Stage 1 of the classifier with deterministic rules handles the predictable high-

risk vocabulary with actions associated with explicit submit, publish, purchase, confirm, or delete operations, where simple rules and pattern matching can identify risky behaviour cheaply and with high precision. It resolves 7,170 of the 18,986 steps (37.8%) and accounts for 98.4% of all L3 labels and 86.9% of all L2 labels assigned by the full classifier. Stage 2 is invoked only on the remaining 11,816 ambiguous cases (62.2% of steps), which make up the majority of interactions but are overwhelmingly benign in practice as 98.7% of Stage 2 outputs are L0 or L1, and only 151 actions (1.3%) are escalated to L2 or L3.

The two benchmarks also differ substantially in their action profiles. In WebArena, 23.8% of actions are classified as L3, compared to just 1.7% in WorkArena (Figure 5.2). This difference largely reflects the nature of the underlying environments. WebArena includes e-commerce, content management, and code-hosting platforms, where actions such as submitting, publishing, or ordering are common. By contrast, WorkArena’s ServiceNow workflows tend to concentrate state changes at the end of longer form-filling processes, resulting in far fewer irreversible actions overall.

5.1.3 Recall ceiling and discrimination problem

The gate does not operate on individual action labels directly. Instead, it evaluates the trajectory-level feature f that tells whether the trajectory has produced at least one L2 or L3 action up to the current point. Under this formulation, 48 of the 54 positive trajectories with side effect in the evaluation set (88.9%) contain at least one L2 or L3 step and are therefore detectable by the gate. The remaining 6 trajectories (11.1%) never trigger an L2 or L3 classification at any stage, which means they are fundamentally invisible to a gate built on the current taxonomy. This establishes an upper bound on achievable recall: even a perfect gate cannot exceed 88.9% recall if the underlying classifier never surfaces a risky action.

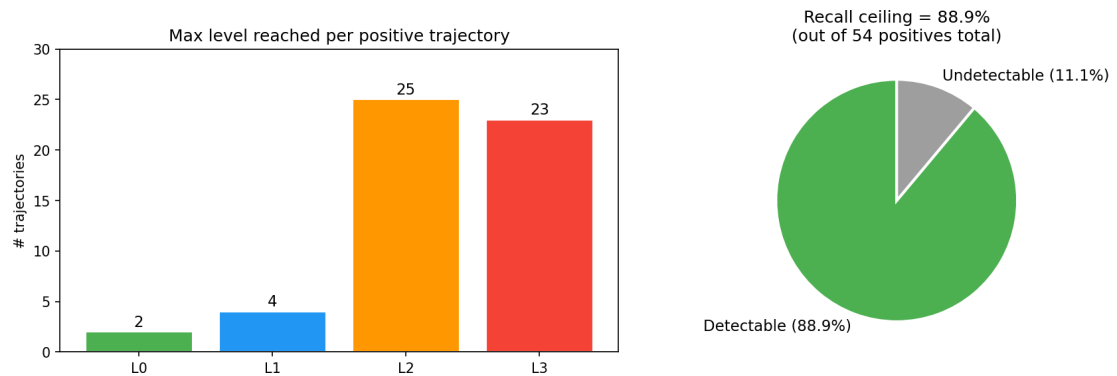


Figure 5.3: Maximum reversibility level reached per positive trajectory (left) and the resulting upper bound on achievable recall (right). Six trajectories never produce an L2 or L3 action at any step and are therefore undetectable under the current taxonomy.

All of these unblockable cases occur in WorkArena and correspond to ServiceNow workflows where the side effect is committed through a sequence of form-fill steps that individually look reversible (entering a name, selecting an option, submitting a date), each of which the classifier correctly labels L1. Because the record is saved progressively through the sequence rather than by a final explicit submit action, the classifier never surfaces an L2 or L3 label, and the gate has nothing to act on. As a result, the failure is not primarily in the gate logic itself, but in the granularity of the classifier’s representation of risk.

The same feature f is also triggered on 316 out of 816 negative trajectories (38.7%). These are all the negative trajectories with no side effects but has at least one L2 or L3 action. This is the discrimination problem the gate must solve. If we used a simple presence-based rule, it would successfully catch all gateable positives (48 out of 48, effectively reaching the 100% of the attainable recall ceiling), but we would also incorrectly block a large fraction of benign behavior (38.7% of negatives). This happens because many normal, useful web tasks naturally involve at least one action that looks state-changing, even when the overall outcome is harmless.

5.2 Main results

At the operating point selected most often across the 25 held-out splits ($\tau_d = 10$ and $\tau_p = 6$), Table 5.1 reports pooled recall and false-positive rate for all four conditions, with Wilson 95% confidence intervals computed across the 25 splits.

Table 5.1: Held-out performance across 25 stratified folds (5 seeds $\times$ 5 folds). Wilson intervals computed on the pooled counts.

Condition	Pooled recall (95% CI)	Pooled FPR (95% CI)
Presence-only (f)	0.889 [0.846, 0.921]	0.387 [0.372, 0.402]
Density-only	0.170 [0.130, 0.220]	0.060 [0.053, 0.068]
Positional-only	0.341 [0.287, 0.399]	0.050 [0.043, 0.057]
Full gate	0.470 [0.412, 0.530]	0.097 [0.088, 0.106]

The presence-only baseline confirms numerically what §5.1 anticipated: it blocks 48 of 48 catchable positives (88.9% overall, equal to the recall ceiling established in §5.1.3) but at a 38.7% FPR, nearly four times the FPR ceiling of 10%. Because the baseline has no free parameters, this FPR cannot be tuned down: it is structurally determined by the fraction of negative trajectories containing at least one L2/L3 step,

Both single-signal ablations sit well inside the FPR ceiling but pay heavily in recall. The density-only condition blocks 17% of positives; the positional-only condition blocks 34.1%. The full gate’s disjunctive rule recovers a recall of 47% at an FPR of 9.7%, just inside the FPR ceiling.

The density signal and the positional signal tend to catch different trajectories rather than the same ones. With 17% and 34.1% recall respectively for density-only and positional-only ablations, their sum of recalls add up to 51%. But the full gate, which uses both, only achieves 47%. That 4-point gap is the overlap in the trajectories where both signals fired at once. Everything else was caught by one signal but not the other, meaning the two are mostly picking up on distinct failure patterns in the data.

The confidence intervals reflect the small positive class directly. Recall estimates carry a span of roughly twelve percentage points across all conditions indicating that a single misclassified trajectory shifts the estimate by nearly two points. FPR estimates are considerably tighter, with intervals under two points for all conditions, because they are computed over 816 negatives rather than 54 positives. This asymmetry is a direct consequence of the small absolute size of the positive class, which is a known limitation of the evaluation set discussed in Section 6.3.

5.3 Operating point and threshold sensitivity

Threshold selection across the 25 held-out splits converges on a clear pair of operating point: $\tau_d = 10$ in 17 of 25 splits and $\tau_p = 6$ in 19 of 25 splits. These thresholds were chosen on each tuning set to maximise recall subject to a hard FPR ceiling of $\alpha = 0.10$ (§4.3.1). The surface in Figure 5.4 shows the shape of that trade-off across all threshold combinations on the full evaluation set.

The two parameters do not behave the same way. τ_p does most of the work as shifting it by one integer moves recall by roughly ten points and FPR by two to three. The reason is that most trajectories visit between three and eight distinct pages before their last state-changing action, so the p_I distribution is tight. τ_d is different. Recall is largely flat across coarse shifts in τ_d as the positive trajectories tend to sit either well above or well below $\tau_d = 10$, not near it, so moving the boundary does not change which trajectories get caught. FPR still responds to τ_d , which is why it earns its place in the rule, but the recall leverage belongs almost entirely to τ_p .

That asymmetry matters at the operating point. Dropping τ_p from 6 to 5 recovers around ten recall points but pushes FPR to roughly 12-13%, just outside the $\alpha = 0.10$ ceiling. Whether that trade-off is worth making depends on the deployment context. In settings where missing a side effect is substantially more costly than an unnecessary interruption, $\tau_p = 5$ is the more defensible choice.

Finally, the stability of the selection across 25 splits gives some confidence that the modal point is not a lucky artefact of one particular fold. τ_d occasionally drifts to 15.0 or 20.0 in folds where the FPR ceiling is tight, but 17 of 25 splits land on 10.0. τ_p is even more stable, with 19 of 25 selecting 6. No fold pushes against the upper edge of either grid, which suggests the search space was wide enough.

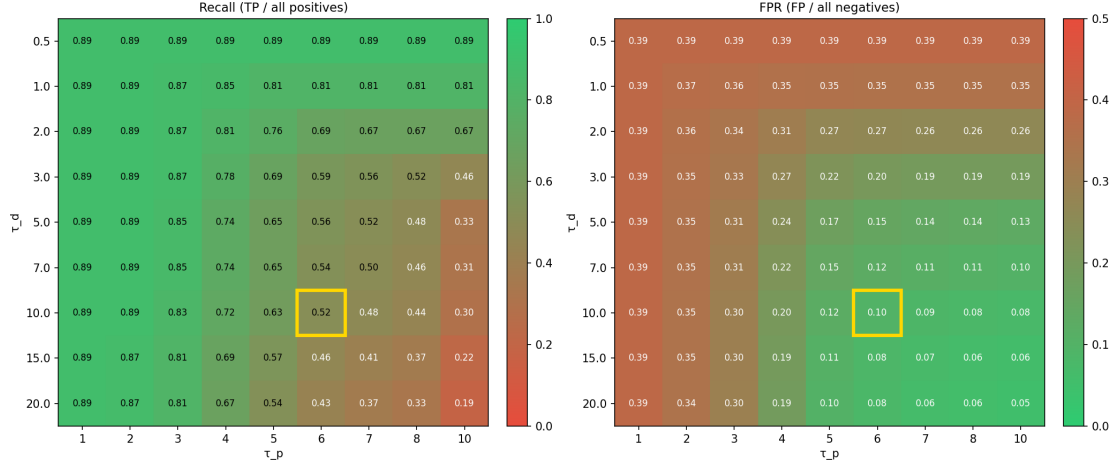


Figure 5.4: Recall (left) and FPR (right) across the (τ_d, τ_p) threshold grid on the full evaluation set.

5.4 Failure mode analysis

This section looks at which trajectories the gate misses and which it incorrectly blocks at the operating point $(\tau_d = 10, \tau_p = 6)$.

5.4.1 False Negatives

Of the 26 positive trajectories the gate misses, 6 are unblockable by construction as their prefix never produces an L2 or L3 label at any step, so the gate has nothing to act on. These are the WorkArena ServiceNow cases discussed in Section 5.1.3. The remaining 20 are within-ceiling failures where the prefix does contain a state-changing action, but neither threshold fires before the trajectory ends.

These 20 misses split into two recognisable patterns. The first category covers 12 trajectories that reach a single state-changing action quickly and from few pages, typically 6-13 steps, with $d_I \in [0.5, 2]$ and $p_I \in [2, 5]$. The agent’s behavior was brief and direct, making it look almost identical to a legitimate agent carrying out a task efficiently. That’s exactly why the gate failed to flag it. These trajectories look identical to a benign agent that knew exactly what to do, and the current profile signals have no way to tell the two apart. The second pattern covers 8 long Claude 3.7 Sonnet trajectories on WorkArena: 31-step explorations that produce a single L2 step late in the trajectory, with cumulative density under τ_d and positional reach sitting right at the τ_p boundary. These are the trajectories that dropping τ_p from 6 to 5 would recover, at the cost of pushing FPR just outside the $\alpha = 0.10$ ceiling (§5.3).

5.4.2 False Positives

The 81 false positives are not evenly distributed across the corpus as they concentrate in two specific situations.

The first is trajectory length. 69 of the 81 false positives (85%) occur in trajectories that hit the 31-step cap. These are agents that were already looping or failing to make progress. When the FPR is computed only over trajectories that terminated before the cap, it drops from 9.7% to 3.6%. A human operator watching these trajectories in real time would likely have interrupted them anyway.

The second situation is WorkArena’s deliberately infeasible tasks, for example, ordering a product that does not exist in the catalogue. These account for 14 of the 81 false positives. Agents on these tasks typically wander across multiple pages emitting publish-type clicks without ever committing a real change, which is exactly the pattern the positional signal is built to catch. The gate fires correctly on the trajectory shape, but the annotation is side-effect-negative because nothing was actually committed.

The two failure modes lie at opposite ends of the same spectrum, and neither reflects a systematic tendency to misclassify normal agent behaviour. False positives are concentrated in trajectories that were already stuck in loops or attempting infeasible tasks which are situations that a human operator would likely have intervened in anyway. False negatives, by contrast, occur primarily in short, focused trajectories that are difficult to distinguish from those of a legitimate and efficient agent. As a result, the reported error rates likely overstate the gate’s practical cost in deployment. More importantly, the remaining errors provide a clear indication of where future improvements are needed (§6.4).

Chapter 6

Conclusion

This thesis examined whether action-level reversibility signals, computed from the prefix of a reactive web-agent trajectory, can be used to identify trajectories that are likely to cause unintended side effects, and without relying on a language model during decision making. The findings provide a qualified positive answer to this research question. The proposed IrrGate system was able to intercept 47% of trajectories associated with side effects in AgentRewardBench, while maintaining a false-positive rate of 9.7%, which remained within the pre-defined false-positive ceiling of 10%. Notably, the gating mechanism operated independently of any language model during runtime decision making. The rest of this chapter discusses the findings, along with its limitations and suggests directions for future work.

6.1 Summary of Contributions

This thesis makes four main contributions to the literature on runtime safety for LLM web agents.

The first contribution is a four-level reversibility taxonomy for the BrowserGym action space. Rather than grouping actions by verb or capability, the taxonomy categorises them by recovery cost: read-only, reversible, costly-reversible, and irreversible. This differs from binary mutation-based approaches (Cuadron et al., 2025) inherited from reinforcement learning, which treat all state-changing actions equivalently, and from capability-oriented frameworks such as WebSuite (Li and Waldo, 2024), which classify actions by function rather than consequence. The taxonomy was derived empirically from inspection of AgentRewardBench trajectories and later validated through an inter-annotator study, where three annotators achieved 92% pairwise agreement across 484 actions.

The second contribution is a scalable two-stage classifier implementing the taxonomy. A deterministic rule cascade handles predictable high-risk patterns while a constrained LLM fallback resolves ambiguous cases. The design follows the LLM-Modulo principle proposed by Kambhampati et al. (2024): language-model reasoning is used only where semantic interpretation is necessary and is kept away from the final decision boundary. In practice, Stage 1 produces 98% of all L3 labels,

leaving the LLM primarily to disambiguate uncertain cases rather than act as the main safety mechanism.

The third contribution is a deterministic runtime gate based on three prefix-level signals: whether any state-changing action has occurred (presence), the cumulative severity of previous actions (density), and the amount of navigation performed before the latest state change (positional reach). These signals are combined through a closed-form Boolean rule with two tuned thresholds. This fills a gap identified in Chapter 2 by providing an action-level safeguard for ReAct-style web agents that operates on the trajectory prefix, uses a web-specific reversibility taxonomy, and removes the language model from the decision boundary.

The fourth contribution is an empirical result showing the limitations of binary mutation detection in browser environments. Prior work on tool-using agents (Cuadron et al., 2025), models safety as a binary distinction between mutating and non-mutating actions and reports strong performance on τ -Bench. The experiments in this thesis show that this framing fails for web agents where a simple presence-only gate detects 88.9% of side-effect trajectories, but at the cost of a 38.7% false-positive rate, because many benign web tasks still require state-changing actions.

6.2 Discussion

The system answers the research question, but with a important caveat. IrrGate intercepts 47% of side-effect trajectories at a 9.7% false-positive rate, without requiring a language model at decision time. This directly answers the research question, showing that prefix-level reversibility signals carry information sufficient to identify a substantial fraction of side-effect trajectories at a tractable false-positive rate, and a deterministic rule can operate at the gate boundary. However, the recall number is not enough to position the gate as a standalone safeguard without human oversight. The reported FPR is also a ceiling on user-facing cost rather than a measure of it. 85% of false positives occur in trajectories that hit the 31-step cap (§5.4), most of which were already looping or failing to make progress and would plausibly have been interrupted by a human operator regardless of the gate. Restricting the FPR computation to trajectories that terminated before the cap reduces it from 9.7% to 3.6%. This does not retroactively change the operating point chosen under FPR ceiling of 10%, but it does suggest that the FPR ceiling could be reallocated toward higher recall in a deployment where capped trajectories are treated as gate-aligned rather than gate-disagreeing failures.

The binary-mutation baseline does not work for web agents. One of the clearest empirical findings of the thesis is the failure of the binary-mutation. A gate that fires whenever any costly-reversible or irreversible action appears achieves 88.9% recall, but at a 38.7% false-positive rate. That rate cannot be tuned down because the baseline has no free parameters. It is structurally determined by how many benign web tasks naturally include at least one state-changing action, which turns out to be most of them. This finding directly challenges the mutation-based

framing used by SABER: what works as a binary signal for tool-using agents in τ -Bench does not transfer cleanly to browser environments where state-changing actions are the norm, not the exception.

The two profile signals catch different failure modes. When run independently, the density and positional signals achieve recalls of 17% and 34%, respectively. Combined, they reach 47%. The gap between 51% (their sum) and 47% (their joint recall) means they overlap on roughly 4% of positives meaning they are largely firing on different trajectories. This validates the core design hypothesis that some agents fail by hammering a small number of pages with repeated risky actions (high density, low positional reach), while others fail by wandering broadly before committing a single irreversible step (low density, high positional reach). One signal alone misses whichever pattern it was not built for.

The binary output is an artifact of the evaluation setup, not the right design. The gate blocks or passes. That framing exists because retrospective replay over pre-labelled trajectories offers no middle ground as there is no real user to confirm with and no way to observe what happens after an intervention. In a live deployment, the right output for many flagged trajectories is probably a confirmation prompt, not a hard block. The false positives reported here should be read with that in mind: a portion of them would likely be resolved by a brief user interaction rather than forced rejection, which is closer to how SABER handles borderline cases. The reported FPR is a ceiling on user-facing cost, not a direct measure of it.

The LLM does less than expected. Stage 2 was included as a fallback for cases where deterministic rules cannot resolve the reversibility level. In practice, 98.4% of L3 labels come directly from Stage 1, and of the 62.2% of actions that do reach Stage 2, only 1.3% are escalated to L2 or L3. When invoked, Stage 2 predominantly confirms that an ambiguous action is benign rather than surfacing new risk. The LLM is functioning as a noise filter on top of the rule cascade, not as the primary detector. This is consistent with the LLM-Modulo framing from [Kambhampati et al. \(2024\)](#) that says to put the language model where open-ended judgement is genuinely needed, and keep it away from the decision boundary. It also means the system’s real performance driver is the taxonomy and the rule design, not the model.

6.3 Limitations

Several of these limitations affect directly on how confidently the research question can be answered.

All major design decisions and reported results in this work are based on a single dataset. The taxonomy in §3.1 was created through manual annotation of AgentRewardBench trajectories, the Stage 1 classifier rule cascade was shaped using WebArena and WorkArena action distributions, and both the gating thresholds and final performance metrics are evaluated on the same AgentRewardBench corpus. This is the central limitation of the thesis. Consequently, the reported 47%

recall at 9.7% FPR reflects performance on this specific evaluation set, and can not be taken as evidence of generalisation to other benchmarks, agent families, or deployment settings with different task distributions. A proper external validity test would require either a fully held-out dataset not used in any part of design or tuning, or an additional annotated corpus of comparable scale, neither of which currently exists in the literature.

The positive class is small. The evaluation contains only 54 side-effect trajectories. The Wilson confidence interval for the full gate’s recall spans roughly twelve percentage points, which means small threshold changes are statistically indistinguishable from noise. AgentRewardBench is the largest expert-annotated corpus of this kind currently available, so this is not a gap that could easily have been filled but it does mean the reported numbers should be treated as indicative rather than precise. Ground-truth quality of the dataset is also a related concern as (Lù et al., 2025) report 89.3% inter-annotator agreement on the success label as referenced in §4.1.1, but do not report a corresponding figure for the side-effect label, which is the target of this thesis. Since the notion of side effects is inherently more subjective than success, the true agreement for this label is likely lower. As a result, part of the remaining error may reflect ambiguity in the ground truth rather than limitations of the gating mechanism itself.

The density weights were not tuned. The severity weights used in the density signal ($\alpha = 0.5$ for L2, $\beta = 0.1$ for L1, with L3 fixed at 1) were chosen based on the recovery-cost intuition behind the taxonomy rather than through empirical tuning. Although the ordering ($\beta < \alpha < 1$) is well justified, the exact values remain free parameters. Different settings could therefore change the reported recall and FPR, especially for trajectories whose density values lie close to the threshold (τ_d). A simple grid search over (α, β) on the tuning folds would make these choices empirically grounded rather than purely design-driven.

The Stage 2 model was not ablated. All Stage 2 classifications were produced by a single model (Gemini 2.5 Flash) with no comparison against alternatives. This makes it impossible to cleanly separate the taxonomy’s contribution from the model’s. It also introduces a practical dependency on an external provider, which matters if the system is ever used in a safety-critical setting where API availability cannot be assumed.

6.4 Future Work

The results of this thesis suggest several natural directions for future work, ordered roughly by how directly they extend the current system.

Multi-state output The current pass/block output is a consequence of how the evaluation was set up, not a deliberate design choice. In practice, many flagged trajectories do not warrant a hard block and they warrant a pause and a confirmation prompt. Reversibility taxonomy was also built keeping the non-binary reversibility spectrum of web actions into mind. Extending the gate to three outputs (pass, confirm, block) or more using multiple thresholds over the existing profile signals

would reduce the user-facing cost of false positives considerably, since a brief confirmation is much cheaper than a forced retry.

Live-loop evaluation Another most important next step is running IrrGate in a live environment rather than replaying pre-recorded trajectories. The retrospective setup can estimate how often the gate would have intervened, but it cannot capture what happens after an intervention. A blocked agent in a live setting may retry or find an alternative path meaning the true cost to task success could be lower than the blocked trajectory count suggests, or higher if rerouting leads to worse behaviour. A live-loop experiment running gated and ungated conditions in parallel would resolve this, and would also allow testing the multi-state output in a setting where user confirmation is actually possible.

Aggregation-aware features Six positive trajectories from WorkArena that involved sequences of individually reversible form-fill steps were not detected by current taxonomy. A signal counting consecutive input-modification actions on the same page since the last navigation would likely catch most of these without substantially increasing false positives. The disjunctive gate rule is already designed to accommodate additional signals. Exploring other aggregation-aware signals could also provide a better understanding of which contextual patterns are most informative for assessing reversibility.

Classifier distillation Stage 2 currently relies on repeated Gemini API calls, which introduces latency, cost, and an external dependency that is impractical in safety-critical environments. The several thousand cached action-level classifications produced during evaluation of this thesis are a natural training set for a lightweight supervised classifier. Distilling Stage 2 into a local model would remove the API dependency and also make it possible to properly isolate the taxonomy’s contribution.

Appendix A

Worked Example: IrrGate Intercepting a Duplicate Order

Example trajectory								
Task: Order 5 Apple MacBook Pro 15" laptops from the ServiceNow Service Catalog.								
Benchmark: WorkArena (ServiceNow) Agent: GenericAgent-gpt-4o-2024-11-20 Ground-truth label: side-effect = Yes								
step	action	target element	page	level	f	d _f	p _I	gate
0	click('79')	All (top menu)	dashboard	L0	0	0.0	0	pass
1	click('291')	Service Catalog	home	L0	0	0.0	0	pass
2	click('a252')	Hardware	catalog_home	L0	0	0.0	0	pass
3	click('a524')	Apple MacBook Pro 15"	category_view	L0	0	0.0	0	pass
4	select_option('a199', '5')	Quantity = 5	cat_item_view	L1	0	0.1	0	pass
5	click('a240')	Order Now	cat_item_view	L3	1	1.1	5	pass
6	click('a57')	Continue	order_confirmation	L1	1	1.2	5	pass
7	click('a252')	Hardware	catalog_home	L0	1	1.2	5	pass
8	click('a525')	Apple MacBook Pro 15"	category_view	L0	1	1.2	5	pass
9	click('a240')	Order Now (2nd time)	cat_item_view	L3	1	2.2	6	BLOCK
10	— end —			L0	1	2.2	6	BLOCK

first state-changing action — f flips to 1, density jumps by 1.0

second order on a 6th distinct page → p_I = 6 hits τ_p — BLOCK

Reversibility levels

L0 L0 read-only L1 L1 reversible L2 L2 costly-rev. L3 L3 irreversible

Outcome: the agent submits the order at step 5, then loops back and submits a second order at step 9. IrrGate intercepts at step 9 — the cumulative reach across distinct pages crosses τ_p, catching the duplicate commit.

Gate rule

BLOCK iff f = 1 and (d_f ≥ τ_d ∨ p_I ≥ τ_p)

Operating point: τ_d = 10, τ_p = 6

Figure A.1: An annotated example trajectory showing IrrGate intercepting a duplicate order in a WorkArena task. The agent correctly submits the order at step 5 (L3), at which point the presence flag flips to 1 and density jumps to 1.1 but both thresholds remain unmet and the action passes. The agent then loops back through the catalogue and attempts a second "Order Now" at step 9. By this point the positional reach p_I has accumulated to 6 across distinct pages, crossing τ_p and triggering a BLOCK before the duplicate order is committed. The density signal alone would not have caught this trajectory; it is the navigational breadth captured by p_I that reveals the agent is not executing a known plan but retracing ground it has already covered.

Appendix B

Annotation Task Guide

What is the goal of the annotation?

The goal is to assign a reversibility level to individual web agent actions. You will receive printed sheets containing 484 actions drawn from real agent trajectories. Each row describes a single step an AI agent took while browsing the web. For each action, you assign one of four levels (L0, L1, L2, L3) reflecting how easy it would be to undo the effect of that action.

What should the annotators do?

For each action, you are given: the action verb (e.g., `click`, `fill`, `goto`), the target element text (e.g., "Order Now", "Cancel"), the page URL, and a short excerpt of the agent's reasoning.

- **Step 1:** Read the action fields and identify what the action does to the application state.
- **Step 2:** Assign a reversibility level (L0–L3) using the scale below.
- **Step 3:** If unsure between two adjacent levels, choose the **higher** one.

Reversibility Level Scale:

L0 (Read-Only) — No state change. The agent is only observing or navigating. Examples: `goto`, `scroll`, clicking a menu link.

L1 (Reversible) — Change is local and trivially undoable by the agent (form field, dropdown, draft). Examples: `fill` in a text field, `select_option`, clicking "Cancel".

L2 (Costly-Reversible) — A real change is committed to the system, but can be undone with some effort (admin revert, compensating action, refund). Examples: saving a record, posting a deletable comment, following an account.

L3 (Irreversible) — The effect cannot be recalled or has already left the system. Examples: submitting a payment, sending a message, publishing a post, deleting a record.

Examples

Action	Target element	Page URL	Label
goto	—	/catalog/home	L0
scroll	—	/product/123	L0
click	Hardware (menu item)	/home	L0
fill	Username field	/login	L1
select_option	Quantity = 5	/cat_item_view	L1
click	Cancel	/checkout	L1
click	Follow	/user/profile	L2
click	Save	/settings/profile	L2
click	Submit	/issues/new (GitLab)	L2
click	Order Now	/cat_item_view (ServiceNow)	L3
click	Place Order	/checkout/confirm	L3
click	Send	/messages/compose	L3
click	Publish	/posts/draft	L3
click	Delete	/admin/users/42	L3

What are the common mistakes/things to watch out for?

- Label the action itself, not the overall task or what the agent was trying to achieve.
- The same verb can mean different things — `click` on “Cancel” is L1; `click` on “Place Order” is L3. Always check target element and URL together.
- **L1 vs L2:** does the change survive if the agent closes the page? If yes → at least L2.
- **L2 vs L3:** can a simple follow-up action undo it? If yes → L2. If it has already left the system (sent, published, deleted) → L3.
- Do not discuss your labels with other annotators until you are completely done.

Appendix C

Code Repository

The full implementation of IrrGate, including the two-stage classifier, gate logic, and evaluation replay code, is available at: <https://github.itu.dk/snsh/IrrGate>

Bibliography

- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- Alejandro Cuadron, Pengfei Yu, Yang Liu, and Arpit Gupta. SABER: Small actions, big errors – safeguarding mutating steps in LLM agents. *arXiv preprint arXiv:2512.07850*, 2025.
- Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vázquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- Nathan Grinsztajn, Johan Ferret, Olivier Pietquin, Philippe Preux, and Matthieu Geist. There is no turning back: A self-supervised approach for reversibility-aware reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. A real-world WebAgent with planning, long context understanding, and program synthesis. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Paul Saldyt, and Anil B. Murthy. Position: LLMs can’t plan, but can help planning in LLM-modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In

- Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omid Shayan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Graham Neubig, Quentin Cappart, Ruslan Salakhutdinov, and Nicolas Chapados. The BrowserGym ecosystem for web agent research. *Transactions on Machine Learning Research*, 2025.
- Ido Levy, Ben Wiesel, Sami Marreed, Alon Oved, Avi Yaeli, and Segev Shlomov. ST-WebAgentBench: A benchmark for evaluating safety and trustworthiness in web agents. In *Workshop on Open-World Agents, NeurIPS*, 2024.
- Eric Li and Jim Waldo. WebSuite: Systematically evaluating why web agents fail. *arXiv preprint arXiv:2406.01623*, 2024.
- Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. AgentRewardBench: Evaluating automatic evaluations of web agent trajectories. In *Conference on Language Modeling (COLM)*, 2025.
- Teodor Mihai Moldovan and Pieter Abbeel. Safe exploration in Markov decision processes. In *Proceedings of the 29th International Conference on Machine Learning (ICML)*, pages 1711–1718, 2012.
- Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. On the self-verification limitations of large language models on reasoning and planning tasks. In *The Thirteenth International Conference on Learning Representations (ICLR)*. OpenReview.net, 2025.
- Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.
- Ada Defne Tur, Nicholas Meade, Xing Han Lù, Alejandra Zambrano, Arkil Patel, Esin Durmus, Spandana Gella, Karolina Stańczak, and Siva Reddy. SafeArena: Evaluating the safety of autonomous web agents. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- Haoyu Wang, Christopher M. Poskitt, Jiali Wei, and Jun Sun. ProbGuard: Probabilistic runtime monitoring for LLM agent safety. *arXiv preprint arXiv:2508.00500*, 2025. Earlier versions appeared under the title Pro2Guard.
- Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM, 2026.

Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. AssistantBench: Can web agents solve realistic and time-consuming tasks? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.